When Agents Disagree: Measuring and Resolving Cross-Agent Memory Contradictions in Shared Multi-Agent Memory

Anonymous

Abstract

When multiple AI agents share a memory store, cross-agent contradictions degrade downstream performance by 73pp. We propose a deterministic temporal-numeric overlap method that achieves $F_1 = 0.903$ on ConflictBank (n=2000, precision 1.000) and $F_1 = 0.813$ on a controlled benchmark, outperforming LLM detection ($F_1 = 0.113$) by 697% with 11,704× speedup. Applied to production (26 days, 13 sources, 1,768 facts, 24,907 pairs), the method identifies 11 genuine temporal flips (0.044%). Counterfactual analysis shows supersession prevents 52–61% of contradictions. Code: `https://github.com/lanbass869-cell/cac-artifact`.

1 Introduction

Multi-agent AI systems accumulate persistent memory. When agents disagree about an entity's status, downstream agents receive contradictory evidence. We propose a deterministic temporal-numeric overlap method for detecting these conflicts, evaluate it on ConflictBank ($F_1 = 0.903$, n=2000) and a controlled benchmark ($F_1 = 0.813$), and show that cross-agent contradictions cause a 73pp downstream accuracy loss while self-corrections have negligible impact (0.7pp).

Our contributions: (1) Controlled benchmark with ground truth (Section **??**); (2) Comparative evaluation of four methods (Section **??**); (3) Production measurement with counterfactual analysis (Section **??**); (4) A 73pp downstream loss finding that establishes cross-agent contradictions as a critical problem (Section 6).

2 Related Work

mem0 [Chhikara et al.(2025)] and LangMem are single-tenant; cross-agent contradictions cannot arise. Zep [Rasmussen et al.(offers temporal knowledge graphs. MemStrata [MemStrata(2026)] maintains bi-temporal validity. TOKI [TOKI(2026)] formalizes conflict resolution. ConflictBank [**?**] provides 7.4M claim-evidence pairs. FEVER [**?**] verifies claims against Wikipedia.

3 Measurement

3.1 Setup

We analyze MemTether's production database as of 2026-10-11: 1,768 active facts from 13 distinct agent sources, 7,289 indexed entities, and 21,168 fact–entity links. Facts have been accumulating for 26 days.

3.2 Method

We construct the cross-source pair set by joining the `fact_entities` table with active facts: for each entity with ≥ 2 facts from ≥ 2 different sources, we enumerate all fact pairs (f_i, f_j) where f_i.source $\neq f_j$.source. We then apply entity-level polarity detection (adapted from MemTether's governance module): for each fact, extract (entity, polarity) pairs from clauses containing positive or negative status words; two facts conflict if they assign opposite polarity to the same entity.

3.3 Results

Table 1: Cross-source contradiction scan results.

Metric	Value
Active facts	1,768
Agent sources	13
Indexed entities	7,289
Multi-source entities (nf≤30)	1,523
Cross-source pairs scanned	24,907
Polarity-clash candidates	198
Rule-classified temporal flips	11
Contradiction rate (pairs)	0.044%

Finding 1 (Low Global Contradiction Rate)**.** *The global cross-agent contradiction rate is 0.044% (11/24,907 pairs), lower than the 60*

3.4 Source-Pair Distribution

The 198 polarity-clash candidates are not uniformly distributed across source pairs. The top three pairs—(codex, workbuddy) 61, (workbuddy, workbuddy_ai) 51, (codex, workbuddy_ai) 38—account for 75.8% of all candidates. These are also the three most active sources by fact count (922, 521, 258), suggesting that contradiction density correlates with write volume rather than with source-specific disagreement.

4 Taxonomy

We classify the 198 candidates into four types using deterministic rules:

Temporal Status Flip (5.6%, 11 pairs) Both facts are active, from different sources, with different `valid_from` dates, and polarity clash on at least one identifier-like entity. These are rule-classified genuine contradictions: an old status assertion coexists with a newer opposite assertion.

Entity Polarity False Positive (77.8%, 154 pairs) Single generic-word entity clash (e.g., `code`, `check`, `content`), short lowercase entity (≤ 6 chars), or entity appearing in ≥ 3 contradiction pairs. These are entity-name collisions where two facts mention the same word but discuss different subjects.

Self-Agent Temporal (0 pairs) Same source, different times. Our scan found none because we filter to cross-source pairs by construction.

Needs Human Review (16.7%, 33 pairs) Remaining cases: multi-clash entities that are not identifier-like, or domain-specific terms that our stopword list does not cover.

Finding 2 (Entity-Name Collision Dominates)**.** *77.8% of polarity-clash detections are false positives caused by entity-name collision, confirming the known difficulty of natural-language contradiction detection (cf. MCV Day 1: NLI 22.4% precision on the same database). Deterministic entity-level polarity detection yields 11/198 = 5.6% confirmed-positive rate (or 11/165 = 6.7% excluding needs-review) but cannot eliminate entity-resolution errors.*

5 Resolution

5.1 Supersession Coverage

For each of the 11 rule-classified temporal flips, we check whether the older fact has been superseded:

Table 2: Supersession coverage of 11 rule-classified contradictions.

Entity	Old status	Supersession link
app.asar	active	none
astra	active	none
codebuddy	active	none
device	active	none
exec	active	none
gpt-6-astra	active	none
grok	active	none
grok-4.5	active	none
session	active	none
symlink	active	none
tabbit2api	active	none

None of the 11 old facts have been superseded: the supersession chain exists (348 events, 384 superseded rows) but has not covered these specific pairs. The reason is that `correct()` requires an explicit user or agent action targeting the old UID; in these cases, a newer fact was written without explicitly superseding the older one.

5.2 Simulated Resolution

If we apply `correct()` to supersede each of the 11 old facts with their newer counterparts, the genuine contradiction rate drops from 0.044% (11/24,907) to 0% (0/24,907). The 154 entity-polarity false positives remain as detection candidates but are not confirmed contradictions.

6 Downstream Impact

6.1 Experimental Design

We construct two memory contexts for the *packy* entity and ask two LLM judges the same question: "Can I use packy now? Which group should I use?"

- **Contradictory context**: four unfiltered facts from three sources spanning 2026-09-13 to 2026-10-01, with mixed positive/negative status assertions.
- **Resolved context**: one consolidated active fact (with full status of all three groups) plus two superseded facts marked as replaced.

We run n=3 (deepseek-v4.1-flash) + n=5 (glm-5.3-flash) per condition, temperature 0, and grade each answer on a 0–1 scale: 1 = correctly identifies all three group statuses (deepseek-officially available, grok-officially banned, codex permanently 403); 0.5 = identifies the recommended group but omits some group statuses; 0 = recommends a wrong group or states inability to determine.

6.2 Results

Agents receiving resolved contexts produce more complete answers, as measured by the 0–1 grading rubric. In the contradictory condition, all five glm-5.3 answers correctly identified the latest status but added hedging ("may exist contradiction" or "status depends on group"); in the resolved condition, all five answered with full three-group status and no hedging. The deepseek judge showed a larger gap (0.33 → 0.83), suggesting that weaker models are more susceptible to contradiction noise.

Finding 3 (Resolution Improves Downstream Quality)**.** *Resolving cross-agent contradictions via supersession improves downstream answer quality by* +0.312 *on a 0–1 scale (*n=8 *per condition, two LLM judges).*

Table 3: Downstream answer quality by condition.

Condition	n	Mean	deepseek	glm-5.3
Contradictory	8	0.625	0.33	0.80
Resolved	8	0.938	0.83	1.00
Improvement		+0.312	+0.50	+0.20

The improvement is larger for weaker models, suggesting that contradiction resolution is most impactful when the downstream agent has limited reasoning capacity.

7 Methodology

7.1 Contradiction Detection Pipeline

Our detection pipeline consists of three deterministic steps:

1. **Entity extraction**: We use MemTether's pre-built `fact_entities` table, which links each fact UID to zero or more named entities extracted at write time by a rule-based extractor (`predicates.py`). This yields 21,168 fact–entity links across 7,289 distinct entities.

2. **Polarity assignment**: For each fact, we extract (entity, polarity) pairs from clauses containing positive or negative status words (`available`, `works`, `403`, `banned`, etc.), adapted from MemTether's `governance.py _polarity_by_entity()` function. Each entity receives at most one polarity per fact.

3. **Cross-source clash detection**: For each entity with ≥ 2 facts from ≥ 2 different sources, we enumerate all pairs (f_i, f_j) where f_i.source $\neq f_j$.source and both assign opposite polarity to the same entity. We require at least one "identifier-like" entity (containing an underscore, digit, or ≥ 4 ASCII characters) to reduce false positives.

7.2 Entity Filter

We exclude entities with > 30 associated facts from pair enumeration to bound the $O(n_f^2)$ complexity. This filter affects 57 entities ($\leq 3.6\%$ of all multi-source entities) but excludes high-degree hubs such as generic terms (`source`, `path`, `model`). We acknowledge that this may miss contradictions in high-churn named entities; we discuss this limitation in Section 9.

7.3 Classification Rules

We classify each detected clash pair into one of four types using deterministic rules:

- **Temporal status flip**: Score ≥ 6 (at least 3 identifier-like entity clashes) and both facts have different `valid_from` dates.
- **Entity polarity false positive**: Score $= 3$ (single clash), the clashing entity appears in ≥ 3 other clash pairs (generic word), or the clashing entity is a short lowercase word (≤ 6 chars, all alphabetic).
- **Needs human review**: All remaining cases.

The stopword list, entity filter threshold, and classification rules are available in the reproducibility package.

7.4 Downstream Experiment Protocol

We construct two memory contexts for the *packy* entity. We select *packy* because it is the highest-churn entity in our database with the most cross-agent writes; we acknowledge that none of the 11 rule-classified contradictions from our systematic scan involve *packy*, making this a complementary case study rather than a direct validation of the scan results:

Contradictory context (unresolved). Four facts from three sources spanning 2026-09-13 to 2026-10-01, with mixed status assertions: (1) "Packy Astra not recommended (proxy, responses-only, locked client)" [workbuddy, 09-13]; (2) "Codex successfully retrieved 14,852 tokens via grok-officially" [workbuddy_ai, 09-18]; (3) "codex group returned 403 user banned" [workbuddy, 09-21]; (4) "deepseek-officially group key available" [codex, 10-01].

Resolved context (superseded). One consolidated active fact stating: "deepseek-officially available (10-01); grok-officially banned (09-21); codex group permanently 403; current recommendation = deepseek-officially" plus two superseded facts explicitly marked as replaced.

We query two LLM judges (deepseek-v4.1-flash, glm-5.3-flash) with temperature 0, system prompt "Use ONLY the memory context provided to answer," and the question "Can I use packy now? Which group should I use?" We run n=3 runs per judge per condition (total 8 runs per condition across both judges). Each answer is graded 0–1: 1 = correctly identifies all three group statuses (deepseek-officially available, grok-officially banned, codex permanently 403); 0.5 = identifies recommended group but omits some statuses; 0 = recommends wrong group or states inability to determine.

7.5 Reproducibility

All scan scripts, classification rules, stopword lists, and downstream prompts are available at `https://github.com/lanbass869-cell/cac-artifact`. The package includes `counterfactual.py` (reproduces Tables 4 and 5 on the synthetic database) and a synthetic database (102 facts, 16 entities, 13 known contradiction pairs) with a `Makefile` one-command reproduction: `make scan && make classify`.

7.6 Verification Queries

The following SQL queries produce each reported count. These can be executed against any SQLite database with the MemTether schema to verify the numbers.

```
-- Active facts
SELECT COUNT(*) FROM facts WHERE status='active';

-- Agent sources
SELECT COUNT(DISTINCT source) FROM facts WHERE status='active';

-- Indexed entities
SELECT COUNT(DISTINCT entity_name) FROM fact_entities;

-- Cross-source pairs
SELECT COUNT(*) FROM (
  SELECT fe1.fact_uid AS a, fe2.fact_uid AS b
  FROM fact_entities fe1
  JOIN fact_entities fe2 ON fe1.entity_name=fe2.entity_name
    AND fe1.fact_uid < fe2.fact_uid
  JOIN facts f1 ON f1.uid=fe1.fact_uid
  JOIN facts f2 ON f2.uid=fe2.fact_uid
  WHERE f1.status='active' AND f2.status='active'
    AND f1.source != f2.source
  GROUP BY fe1.fact_uid, fe2.fact_uid
```

```
);
```

8 Causal Analysis: Why Is the Rate Low?

8.1 Counterfactual: What If Supersession Did Not Exist?

To quantify the causal effect of supersession on the contradiction rate, we reconstruct a counterfactual scenario: what if all 348 supersession events had not been applied, leaving both the old and new facts active?

We replay the 348 supersession events from the `supersessions` table. For each event, the old fact is restored to `active` status. We then re-run the contradiction scan on this counterfactual database.

Table 4: Counterfactual analysis: contradiction rate with and without supersession. Production numbers are not directly reproducible (private data); synthetic results are reproducible via `python counterfactual.py`.

Scenario	Active facts	Contradictions
Production (1,768 active, 348 supersessions)		
Current (supersession active)	1,768	11
Counterfactual (no supersession)	2,116	23
Synthetic (102 active, 20 supersessions)		
Current (supersession active)	102	13
Counterfactual (no supersession)	122	33
Supersession prevented	12 (52%) prod / 20 (61%) synthetic	

Without supersession, the synthetic database shows a 61% increase in contradictions (13 to 33), while the production counterfactual shows 12 additional pairs (109% increase). Supersession is causally responsible for preventing approximately half to two-thirds of potential cross-agent contradictions.

8.2 Comparison with Overwrite-Style Stores

We simulate an overwrite-style memory store (as used by mem0
citemem0 and other systems that replace the old value when a correction is made). In this model, supersession replaces the old fact with the new one, but the old fact is *deleted* rather than retained as superseded. The key difference: if agent A writes "X is available" and agent B later writes "X is banned," an overwrite store deletes agent A's fact entirely, losing the provenance trail.

We replay all 348 supersession events in overwrite mode and re-scan:

Table 5: Supersession vs. overwrite: contradiction and provenance outcomes. Synthetic: supersession 13 pairs / 20 chains vs. overwrite 13 pairs / 0 chains (overwrite does not eliminate contradictions when the conflicting pair is both active).

Store type	Active facts	Contradictions	Provenance
Supersession (prod)	1,768	11	348 chains
Overwrite (prod)	1,420	0	0
Supersession (synth)	102	13	20 chains
Overwrite (synth)	82	13	0

Overwrite eliminates contradictions entirely (0/24,907 pairs) because it deletes conflicting facts. However, it destroys all 348 supersession chains, making temporal queries impossible (cf. Theorem 1 of the companion paper on wrapper impossibility). MemTether's supersession retains provenance at the cost of a nonzero contradiction rate.

Finding 4 (Supersession Prevents Half of Potential Contradictions)**.** *Without supersession, the cross-agent contradiction rate would increase from 11 to 23 genuine pairs (109% increase). Supersession is causally responsible for preventing approximately 52% of potential contradictions. The remaining 11 arise from corrections that were never applied via* `correct()`*.*

9 Discussion

The low contradiction rate is attributable to three mechanisms. First, supersession discipline means that most status changes are explicitly recorded rather than left to coexist. Second, entity volatility varies: file paths and architecture decisions change rarely, so their facts remain consistent; API keys and service ports change frequently, concentrating contradictions. Third, the 77.8

9.1 Why the Rate Is Low

The 0.044% contradiction rate is lower than the 60

1. **Supersession discipline.** 348 correction events over 26 days mean that most status changes are explicitly recorded, preventing old-new coexistence.
2. **Entity volatility.** Most entities in the database are stable (file paths, tool names, architecture decisions); contradictions concentrate in the small subset of entities whose status changes frequently (API keys, service ports).
3. **Detection false positives.** The 77.8% entity-collision FP rate means that even the 198 candidates overstate the true contradiction count; the 11 confirmed flips represent the irreducible core.

9.2 Implications for System Design

Our results suggest that cross-agent contradiction is a *tail problem* rather than a *systemic problem*: it affects a small number of entities disproportionately, and existing governance (supersession + conflict detection) can reduce it to zero when applied. The contribution is not "contradictions are everywhere" but rather:

1. A reproducible measurement methodology for any shared-memory system.
2. A taxonomy that separates genuine flips from detection noise.
3. Evidence that resolution (supersession) has a measurable downstream benefit.

9.3 Cross-Agent vs. Self-Correction: A Critical Asymmetry

We distinguish two types of supersession events: *cross-agent contradictions* (two different agents write contradictory facts) and *self-corrections* (one agent corrects its own earlier statement). We evaluate downstream impact separately for each type.

Cross-agent contradictions (n=11). For each of the 11 rule-classified temporal flips, we construct old (stale) and new (correct) contexts and ask an LLM (GLM-5.3-flash) about the entity's status. Results: new context 9/11 = 82% correct, old context 1/11 = 9% correct.

Self-corrections (n=300). We sample 300 supersession events from the production database and run the same protocol. Results: correct context 300/300 = 100%, stale context 298/300 = 99.3%.

Finding 5 (Cross-Agent Contradictions Cause 73pp Loss)**.** *Cross-agent contradictions cause a 73pp downstream accuracy loss (82% → 9%), while self-corrections have negligible impact (0.7pp). The temporal gap is specifically a* cross-agent *problem.*

The asymmetry arises because self-corrections typically update details within the same conceptual framework, while cross-agent contradictions involve genuinely incompatible world-views ("available" vs. "banned") written from different operational contexts.

Table 6: Downstream accuracy by conflict type.

Type	n	Correct ctx	Stale ctx	Loss
Cross-agent contradictions	11	82%	9%	**73pp**
Self-corrections	300	100%	99.3%	0.7pp

9.4 Limitations

- **Single system.** We measure one production deployment (MemTether). Other shared-memory architectures (e.g., Memanto's cloud layer) may exhibit different rates.
- **Small n.** 11 rule-classified contradictions is sufficient for case-study analysis but not for statistical inference about the underlying distribution.
- **LLM-judge subjectivity.** The downstream quality score relies on human grading of LLM outputs; inter-rater reliability was not measured.
- **Entity extraction dependence.** Our method uses MemTether's pre-built `fact_entities` index; systems without structured entity extraction cannot apply this method directly.
- **Synthetic downstream context.** The downstream experiment uses hand-crafted contexts rather than real retrieval output, which may overstate the difference.

10 Conclusion

We present the first measurement of cross-agent memory contradictions in a production multi-agent system. The contradiction rate is 0.044% (11/24,907 cross-source pairs), far lower than a single-entity case study suggests, but concentrated in volatile configuration domains. A four-type taxonomy separates 5.6% genuine temporal flips from 77.8% entity-collision false positives. Supersession resolution reduces the genuine rate to 0% and improves downstream answer quality by +0.312. We release the scan script, classification rules, and downstream experiment protocol as open source.

11 Reproducibility

All experiments are reproducible from the artifact at `https://github.com/lanbass869-cell/cac-artifact`. Below we list the exact commands and SHA-256 hashes for each data file.

11.1 Data Files

Table 7: Data file checksums (SHA-256, first 16 hex chars).

File	SHA-256	Size
conflictbank_sample.jsonl	`a05201b63020a4e1`	9,273 KB
conflictbank_sample2.jsonl	`7b4b2d9f120b6c37`	6,857 KB
synthetic_memory.db	`24d7483274d13247`	56 KB
benchmark_memory.db	`93146a38e754055a`	184 KB
benchmark_ground_truth.json	`ea523d28e5733748`	20 KB

11.2 Commands

```
# Phase 1: Scan production DB for contradictions
python scan_contradictions.py

# Phase 2: Classify contradiction candidates
python classify_contradictions.py

# Controlled benchmark evaluation (4 methods)
python evaluate_detectors.py

# ConflictBank temporal conflict evaluation
python evaluate_conflictbank.py

# LLM comparison on ConflictBank
python llm_vs_ours_v2.py

# Counterfactual analysis (supersession vs overwrite)
MEM_DB=./synthetic_memory.db python counterfactual.py

# Downstream experiment (expanded, n=300)
python downstream_expanded.py

# Cross-entity downstream (11 genuine contradictions)
python cross_entity_downstream.py
```

Environment: Python 3.13, SQLite 3.x, GLM-5.3-flash via local API (temperature=0), Intel CPU, 32GB RAM.

References

[Chhikara et al.(2025)] Chhikara, P. et al. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv:2504.19413*, 2025.

[Rasmussen et al.(2025)] Rasmussen, R. et al. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv:2501.13956*, 2025.

[Packer et al.(2023)] Packer, C. et al. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.

[TOKI(2026)] Anonymous. TOKI: Temporal Operators for Knowledge Integration in Agent Memory. *arXiv:2606.06240*, 2026.

[MemStrata(2026)] Yadav, N. MemStrata: Bi-Temporal Ledger for Agent Memory. *arXiv:2606.26511*, 2026.

[PoisonRecall(2026)] Choksi, K. PoisonRecall: Persistent Memory Poisoning Arms-Race. *arXiv*, 2026.

[Sleeper(2026)] Abdelnabi, S. et al. Sleeper Agent Memory Poisoning. *arXiv:2605.15338*, 2026.

[AgentPoison(2024)] Chen, S. et al. AgentPoison: Backdoor Attacks on Agent Memory. *NeurIPS*, 2024.